

Electronic mail security

Pretty Good Privacy

- Philip R. Zimmerman is the creator of PGP.
- PGP provides a confidentiality and authentication service that can be used for electronic mail and file storage applications.

Why Is PGP Popular?

- It is available free on a variety of platforms.
- Based on well known algorithms.
- Wide range of applicability
- Not developed or controlled by governmental or standards organizations

Operational Description

- Consist of five services:
 1. **Authentication** (Provided through DSS)
 2. **Confidentiality** (Done thru Symmetric Key Encryption algorithms)
 3. **Compression** (ZIP algorithm)
 4. **E-mail compatibility** (Using Radix 64 Conversion)
 5. **Segmentation** (as used in the OS)

Table 5.1 Summary of PGP Services

Function	Algorithms Used	Description
Digital signature	DSS/SHA or RSA/SHA	A hash code of a message is created using SHA-1. This message digest is encrypted using DSS or RSA with the sender's private key and included with the message.
Message encryption	CAST or IDEA or Three-key Triple DES with Diffie-Hellman or RSA	A message is encrypted using CAST-128 or IDEA or 3DES with a one-time session key generated by the sender. The session key is encrypted using Diffie-Hellman or RSA with the recipient's public key and included with the message.
Compression	ZIP	A message may be compressed, for storage or transmission, using ZIP.
Email compatibility	Radix 64 conversion	To provide transparency for email applications, an encrypted message may be converted to an ASCII string using radix 64 conversion.
Segmentation	—	To accommodate maximum message size limitations, PGP performs segmentation and reassembly.

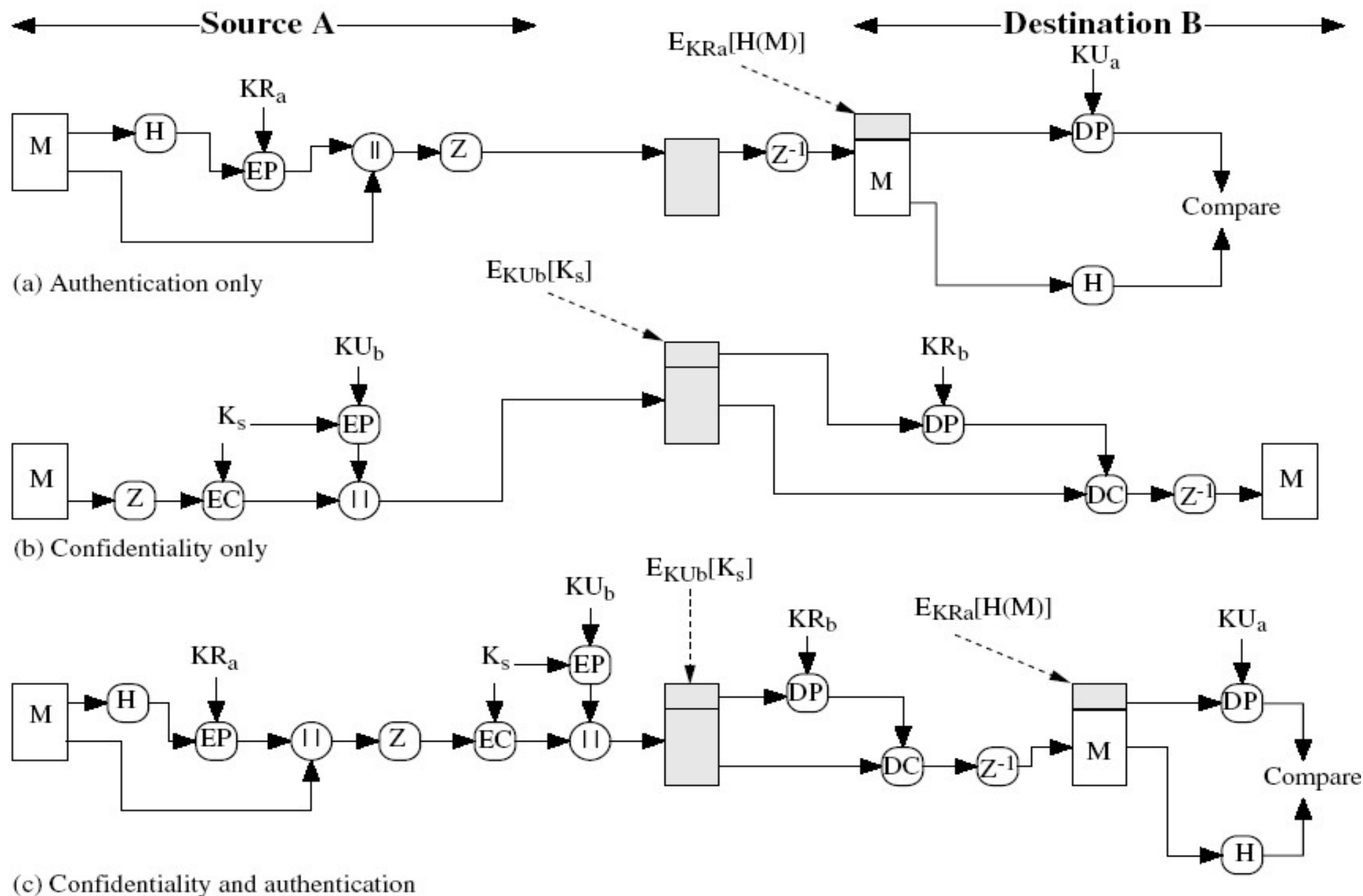
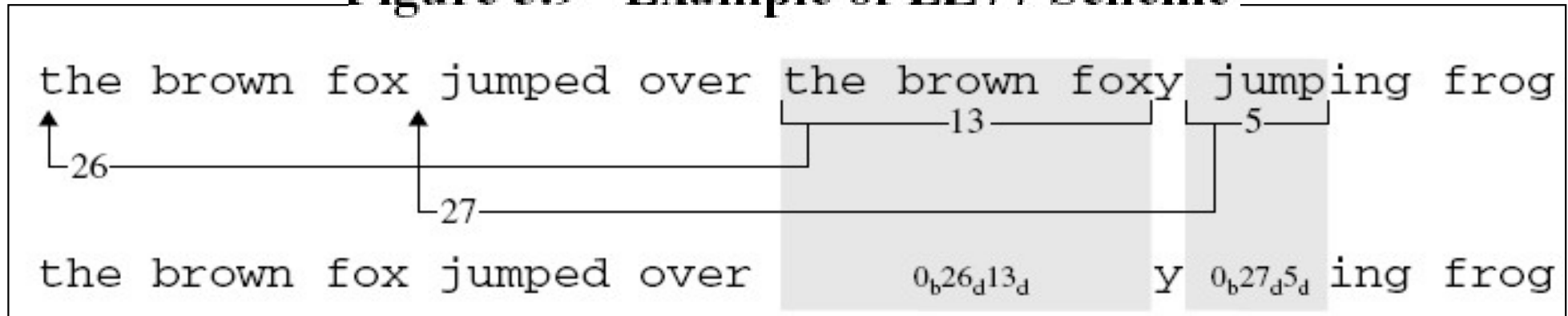


Figure 5.1 PGP Cryptographic Functions

Compression

- PGP compresses the message after applying the signature but before encryption
- **Z** in the earlier diagrams represent the compression function.
- The placement of the compression algorithm is critical.
- The compression algorithm used is ZIP

Figure 5.9 Example of LZ77 Scheme



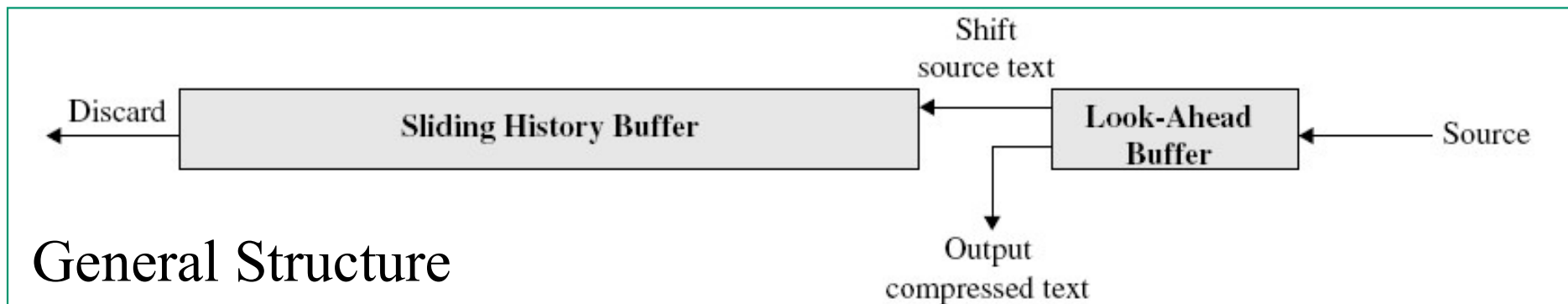
- Octets: 53 (characters including spaces)
- Binary form= 424 bits =(53*8 bits)
- Repeated Sequences: 2
 - Seq1: the brown fox : 13 chars
 - Offset1: 26 characters
 - Seq2: jump : 5 chars
 - Offset2: 27 characters
 - Seq1 code: 0 26 13
 - Seq1 code: 0 27 5
- Encoding format: offset: 8 and length: 4 → 00
offset: 12 and length: 6 → 01

Compressed message: =53-(13+5)=35 9-bit chars and 2 codes
So 35*9+2*14= 343 bits

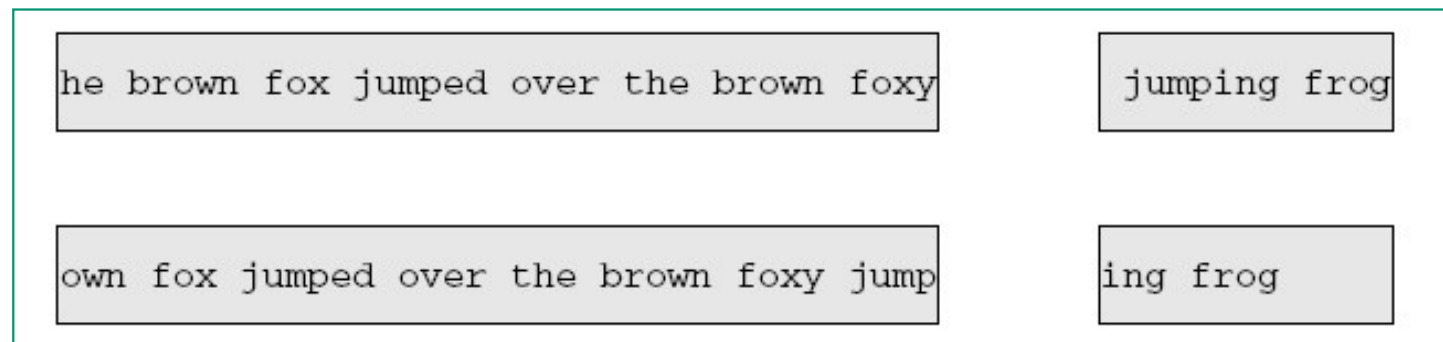
Compression ratio: 1.24 =(424/343)

LZ77 - Ziv and Lempel in 1977

- Based on a portion of the **previous encoded** sequence
- The encoder examines the input sequence through a sliding window
- **The sliding window consists of two parts**
 1. **Search Buffer**
 2. **Look-ahead Buffer**



Example



E-mail Compatibility

- The scheme used is radix-64 conversion.
- **Base64 (Radix-64) encoding schemes** - used to encode binary data that need to be **stored and transferred over media** that are designed to deal with textual data.
- This is to ensure that the data remain intact without modification during transport.
- The use of radix-64 expands the message by 33%.

**Printable
Encoding
of Binary
Data in
Radix 64
Format**

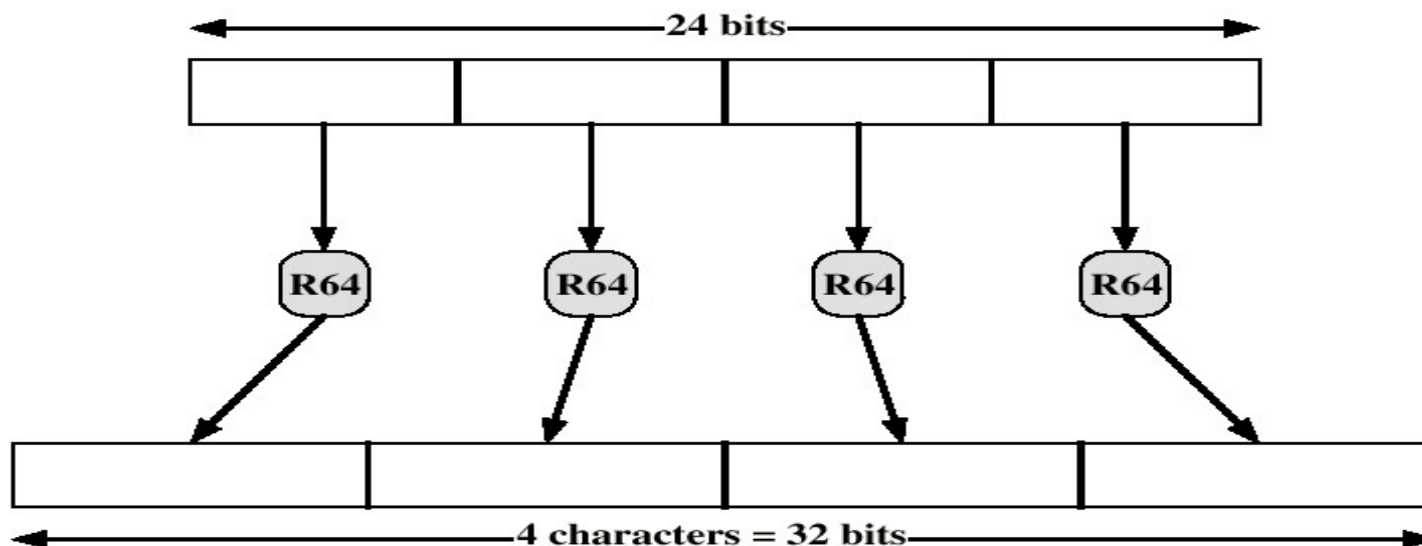


Table 5.9 Radix-64 Encoding

6-bit value	character encoding	6-bit value	character encoding	6-bit value	character encoding	6-bit value	character encoding
0	A	16	Q	32	g	48	w
1	B	17	R	33	h	49	x
2	C	18	S	34	i	50	y
3	D	19	T	35	j	51	z
4	E	20	U	36	k	52	0
5	F	21	V	37	l	53	1
6	G	22	W	38	m	54	2
7	H	23	X	39	n	55	3
8	I	24	Y	40	o	56	4
9	J	25	Z	41	p	57	5
10	K	26	a	42	q	58	6
11	L	27	b	43	r	59	7
12	M	28	c	44	s	60	8
13	N	29	d	45	t	61	9
14	O	30	e	46	u	62	+
15	P	31	f	47	v	63	/
						(pad)	=

i/p : 00100011 01011100 10010001

2 3 5 C 9 1 (in hexadecimal)

arrange this input in blocks of 6 bits:

001000 110101 110010 010001

I 1 y R (in radix-64)

Character	ASCII Value	Binary form
I	73	1001001
1	49	0110001
y	121	1111001
R	82	1010010

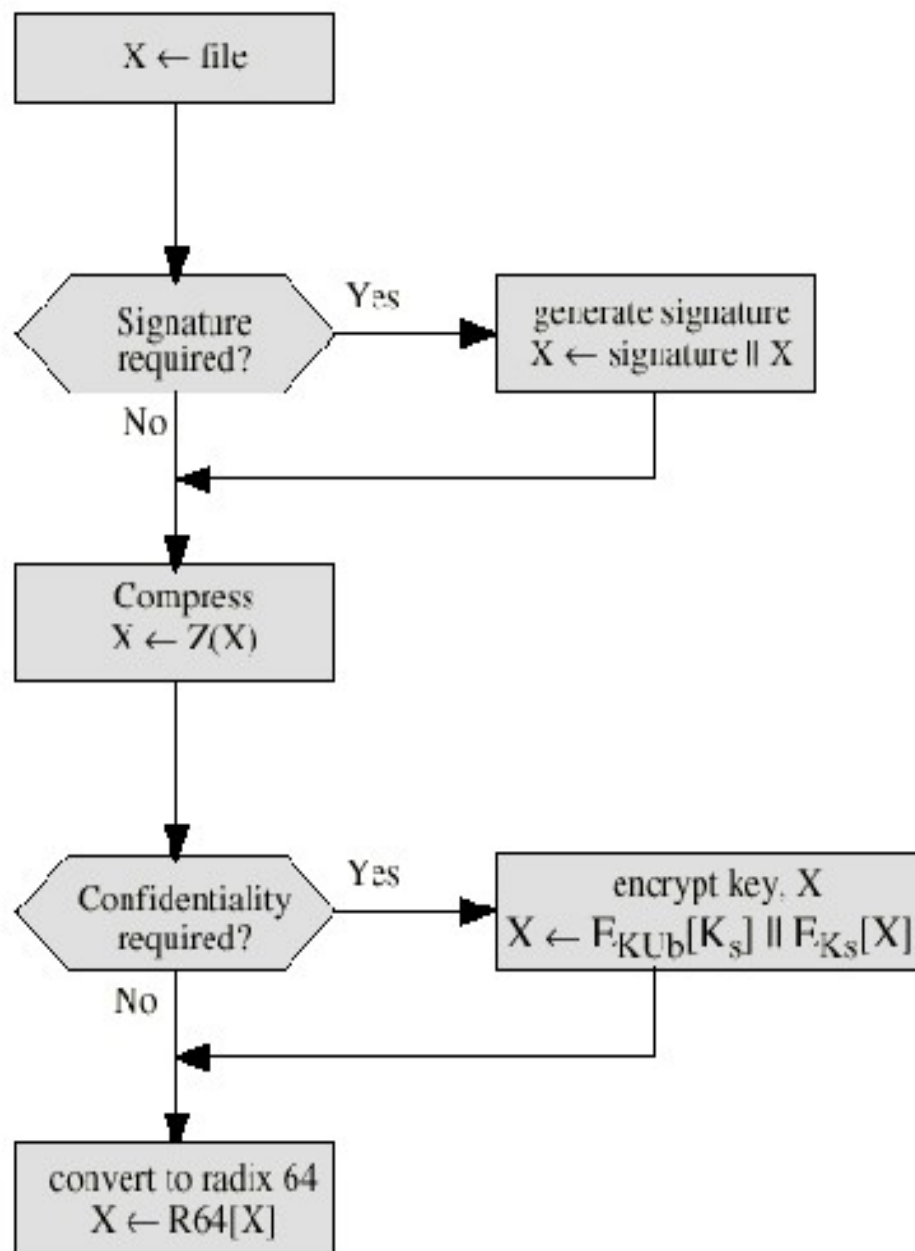
If these characters are stored in 8-bit ASCII format with **parity bit set to zero** , we have

01001001 00110001 01111001 01010010

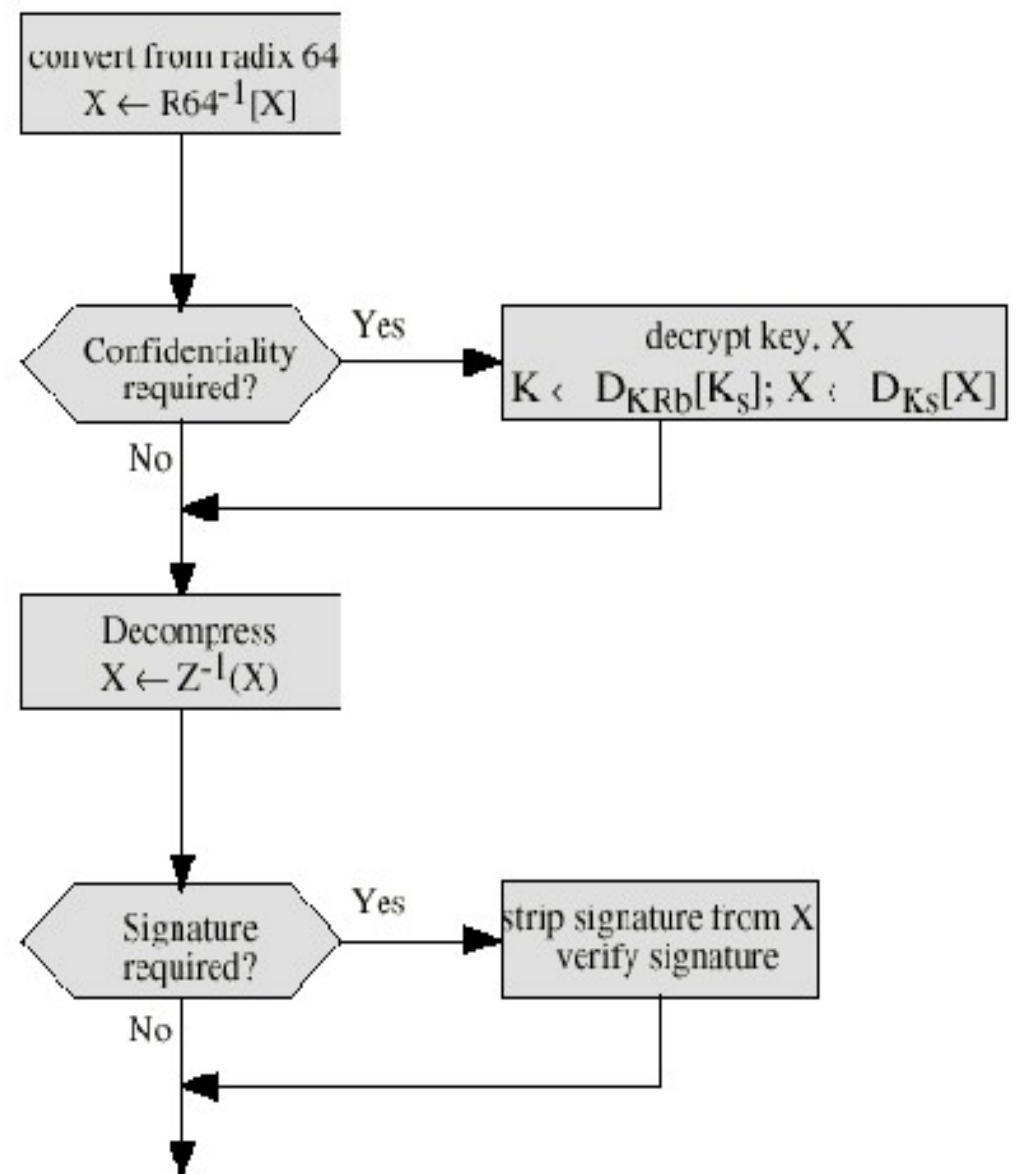
o/p : 4 9 3 1 7 9 5 2

Segmentation and Reassembly

- Often restricted to a maximum message length of 50,000 octets.
- Longer messages must be broken up into segments.
- PGP automatically subdivides a message that is too large.
- The receiver strip of all e-mail headers and reassemble the block.



(a) Generic Transmission Diagram (from A)

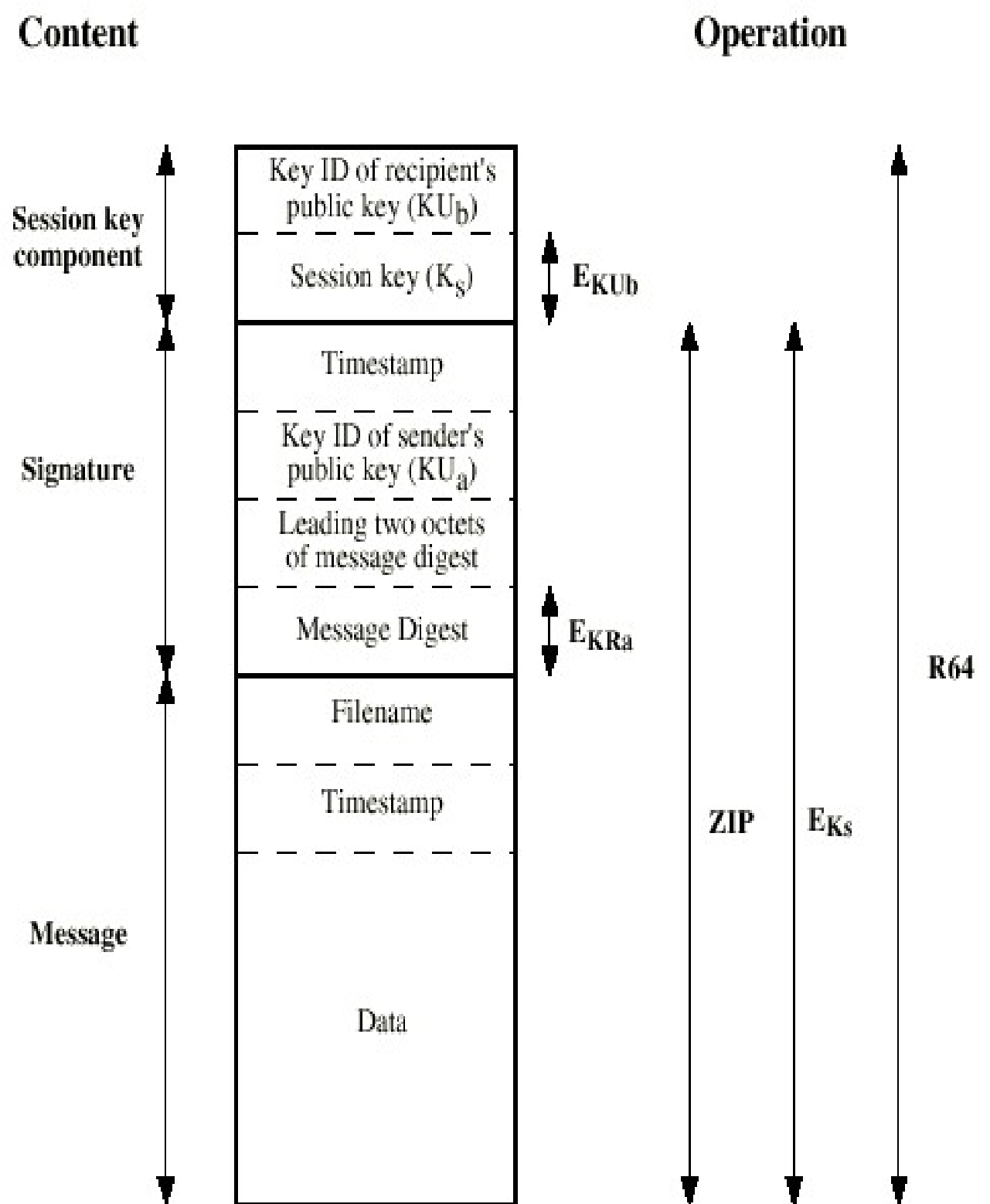


(b) Generic Reception Diagram (to B)

Transmission and Reception of PGP Messages

Format of PGP Message

No. of KeyIDs used
in PGP: 2



Cryptographic Keys and Key Rings

PGP makes use of four types of keys:

1. One-time session symmetric keys
2. Public keys
3. Private keys
4. Passphrase based symmetric keys

- In PGP, session keys and passphrase-based keys are generated on the fly, used once and discarded.
- Public and private keys are persistent and need to be preserved and managed. Recall that a user can have multiple public/private key pairs.

Requirements of the keys

3 separate requirements can be identified w.r.t these keys:

1. A means of generating unpredictable session keys is needed
2. We would like to allow a user to **have multiple public-key/private-key pairs**. As a result there is not a one-to-one correspondence between users and their public keys. Thus, some means is needed for identifying particular keys.
3. Each PGP entity must maintain a file of its own public/private key pairs as well as a file of public keys of correspondents

Key Rings

- No. of Key_IDs are included in any PGP message : 2 (used for authentication and confidentiality)
- These keys need to be stored and organized in a systematic way for efficient and effective use by all parties.
- **Scheme used in PGP** : a pair of data structures at each node,
 - one to store the public/private key pairs owned by that node and
 - one to store the public keys of other users known at this node.
- These data structures are called as the private-key ring and the public key ring.

Terminology of Key Rings

- Each user maintains two key ring data structures:
 1. a private-key ring for his own public/private key pairs, and
 2. a public-key ring for the public keys of correspondents

The **private key ring** is a table of rows containing

1. **Timestamp**: The date/time when this key pair was generated.
2. **Key ID**: The least significant 64 bits of the public key for this entry.
3. **Public Key**: The public-key portion of the pair.
4. **Private key**: The private-key portion of the pair.
5. **User ID**: Typically a user's e-mail address.

Public Key Ring Contents

This is a table of rows containing (among other fields):

1. **Timestamp**: when the entry was generated.
2. **Key ID**: 64 least significant digits of this entry.
3. **Public key**: the public key for the entry.
4. **User ID**: Identifier for the owner of this key.
Multiple IDs may be associated with a single public key.

Private Key Ring

Timestamp	Key ID*	Public Key	Encrypted Private Key	User ID*
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
T_i	$KU_i \bmod 2^{64}$	KU_i	$EH(P_i)[KR_i]$	User i
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.

Public Key Ring

Timestamp	Key ID*	Public Key	Owner Trust	User ID*	Key Legitimacy	Signature(s)	Signature Trust(s)
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
T_i	$KU_i \bmod 2^{64}$	KU_i	trust_flagi	User i	trust_flagi		
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.

* = field used to index table

Figure 5.4 General Structure of Private and Public Key Rings

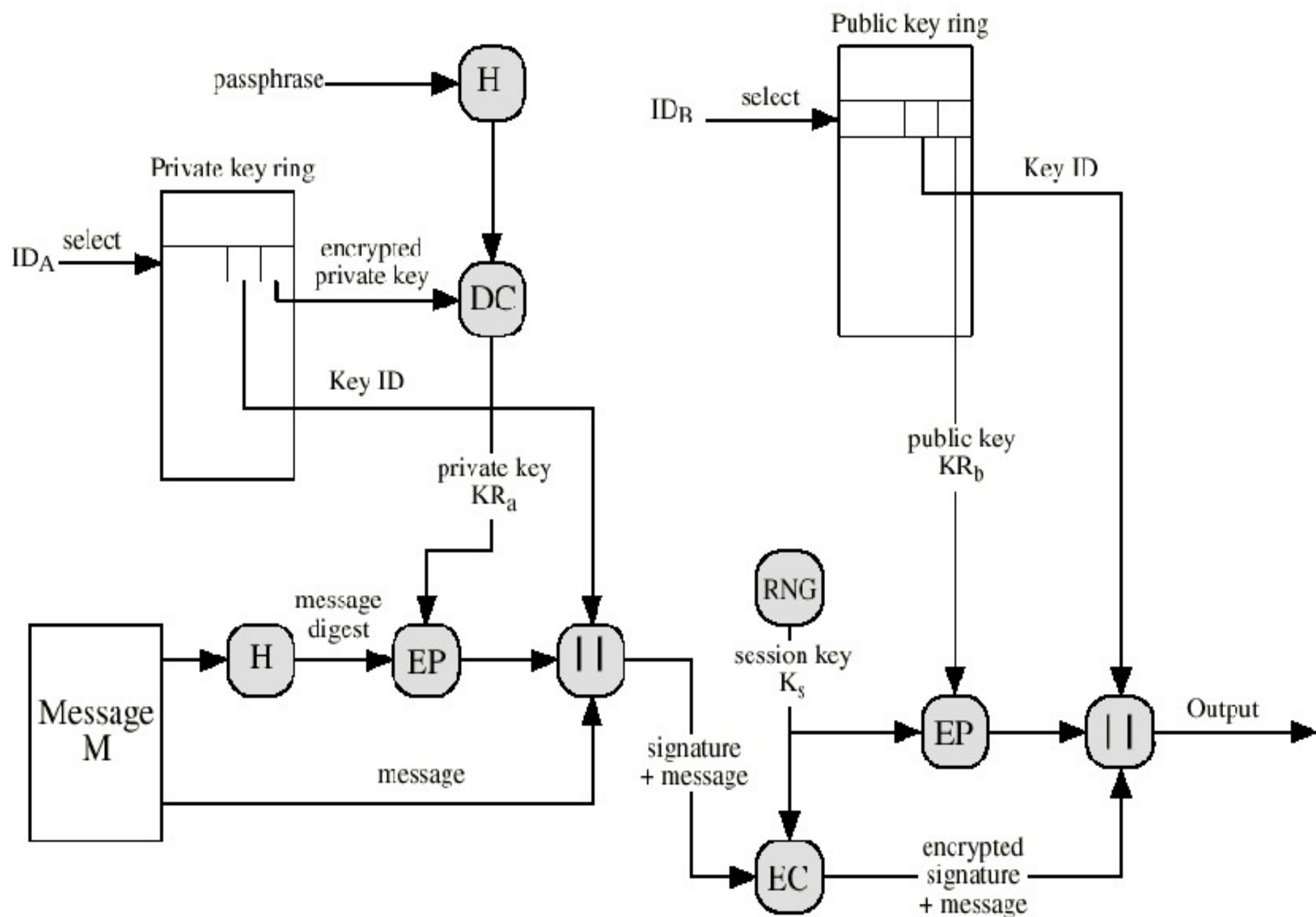


Figure 5.5 PGP Message Generation (from User A to User B; no compression or radix 64 conversion)

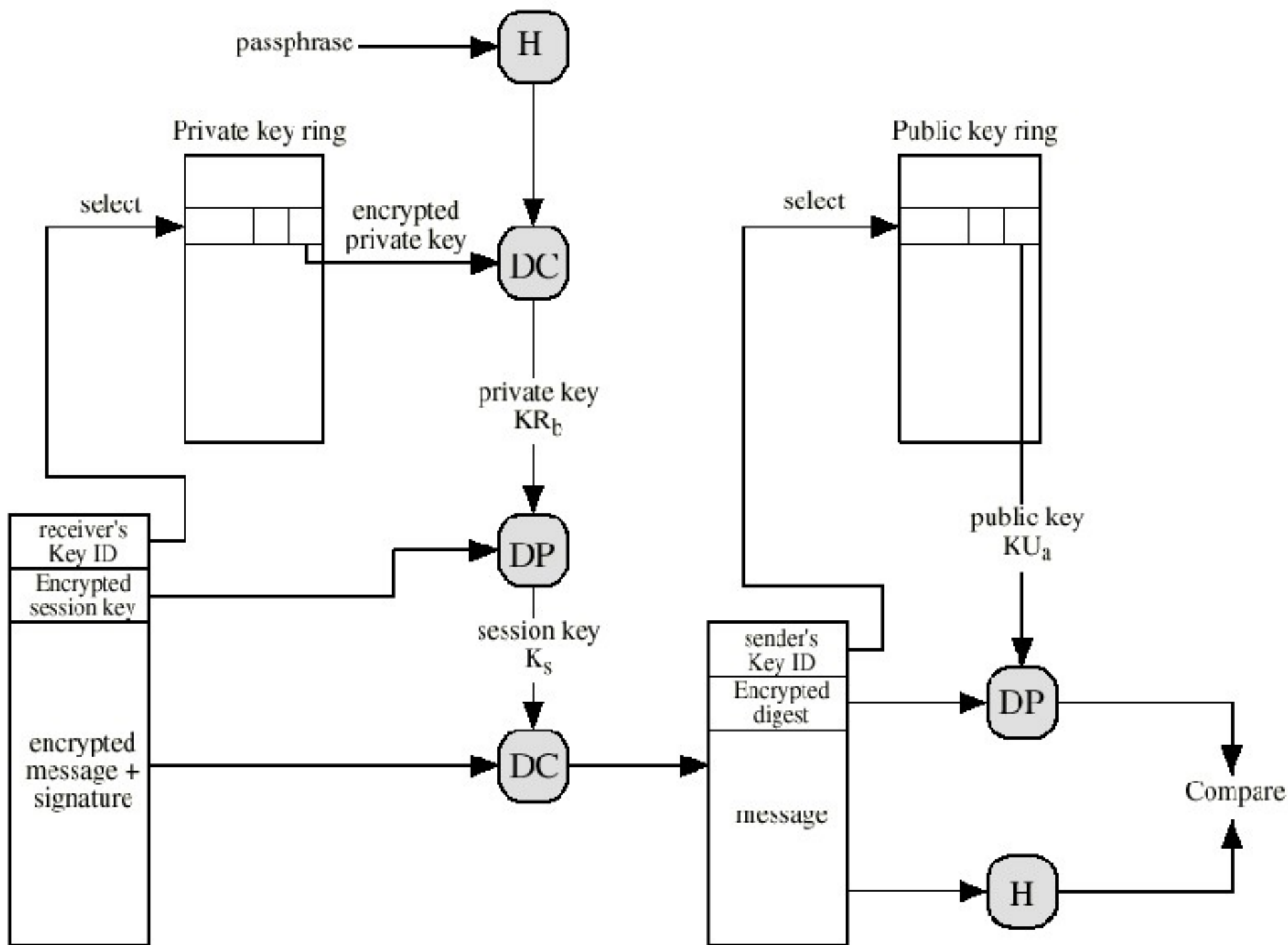


Figure 5.6 PGP Message Reception (from User A to User B; no compression or radix 64 conversion)

Use of Trust

PGP doesn't require certificate , it is optional in PGP.

- Anyone can issue a certificate to anyone else .
- Each user decides whose certificates to trust in authenticating others

Trust can be relayed

In the public key ring

1. Key legitimacy field (trustworthy of binding) : based on signature trust(s)
2. Signature trust field: copy of Owner Trust of this signator
3. Owner trust field: user defined

Public Key Ring

Timestamp	Key ID*	Public Key	Owner Trust	User ID*	Key Legitimacy	Signature(s)	Signature Trust(s)
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
T _i	KU _j mod 2 ⁶⁴	KU _j	trust_flag _j	User _i	trust_flag _j		
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.

Public Key Management:

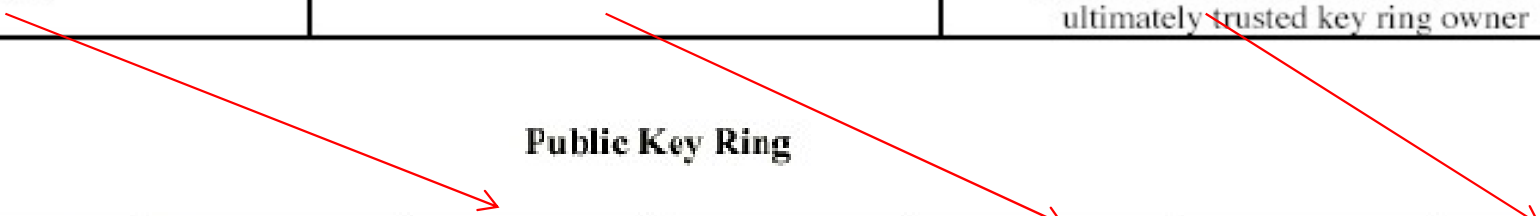
The Use of Trust

Owner trust field	Trust value for the public key which is also enlisted in the Private Keyring of the owner. So <u>Ultimate</u> value is assigned.
Key legitimacy field	• Indicates the extent to which PGP trusts that this is a valid public key for this user. • Determined from ✓ certificates and chains of certificates, ✓ the user's assessment of the trust to be assigned to the key, and ✓ various heuristics for computing trust.
Signature trust field	Indicates the degree to which PGP user trusts the <u>signer</u> to certify public keys.

Table 5.2 Contents of Trust Flag Byte

(a) Trust Assigned to Public-Key Owner (appears after key packet; user defined)	(b) Trust Assigned to Public Key/User ID Pair (appears after User ID packet; computed by PGP)	(c) Trust Assigned to Signature (appears after signature packet; cached copy of OWNERTRUST for this signator)
OWNERTRUST Field — undefined trust — unknown user — usually not trusted to sign other keys — usually trusted to sign other keys — always trusted to sign other keys — this key is present in secret key ring (ultimate trust) BUCKSTOP bit — set if this key appears in secret key ring	KEYLEGIT Field — unknown or undefined trust — key ownership not trusted — marginal trust in key ownership — complete trust in key ownership WARNONLY bit — set if user wants only to be warned when key that is not fully validated is used for encryption	SIGTRUST Field — undefined trust — unknown user — usually not trusted to sign other keys — usually trusted to sign other keys — always trusted to sign other keys — this key is present in secret key ring (ultimate trust) CONTIG bit — set if signature leads up a contiguous trusted certification path back to the ultimately trusted key ring owner

Public Key Ring



Timestamp	Key ID*	Public Key	Owner Trust	User ID*	Key Legitimacy	Signature(s)	Signature Trust(s)
•	•	•	•	•	•	•	•
•	•	•	•	•	•	•	•
•	•	•	•	•	•	•	•
T_i	$KU_i \bmod 2^{64}$	KU_i	trust_flag _i	User i	trust_flag _i		
•	•	•	•	•	•	•	•
•	•	•	•	•	•	•	•
•	•	•	•	•	•	•	•

Revoking Public Keys

- The owner issue a key revocation certificate.
- Normal signature certificate with a revoke indicator.
- Corresponding private key is used to sign the certificate.

S/MIME

- S/MIME: Secure/Multipurpose Internet Mail Extension
- S/MIME will probably emerge as the industry standard.
- PGP for personal e-mail security

Simple Mail Transfer Protocol (SMTP, RFC 822)

- **SMTP Limitations - Can not transmit, or has a problem with:**
 - executable files, or other binary files (jpeg image)
 - "national language" characters (non-ASCII)
 - messages over a certain size
 - ASCII to EBCDIC translation problems
 - lines longer than a certain length (72 to 254 characters)

Figure *MIME*

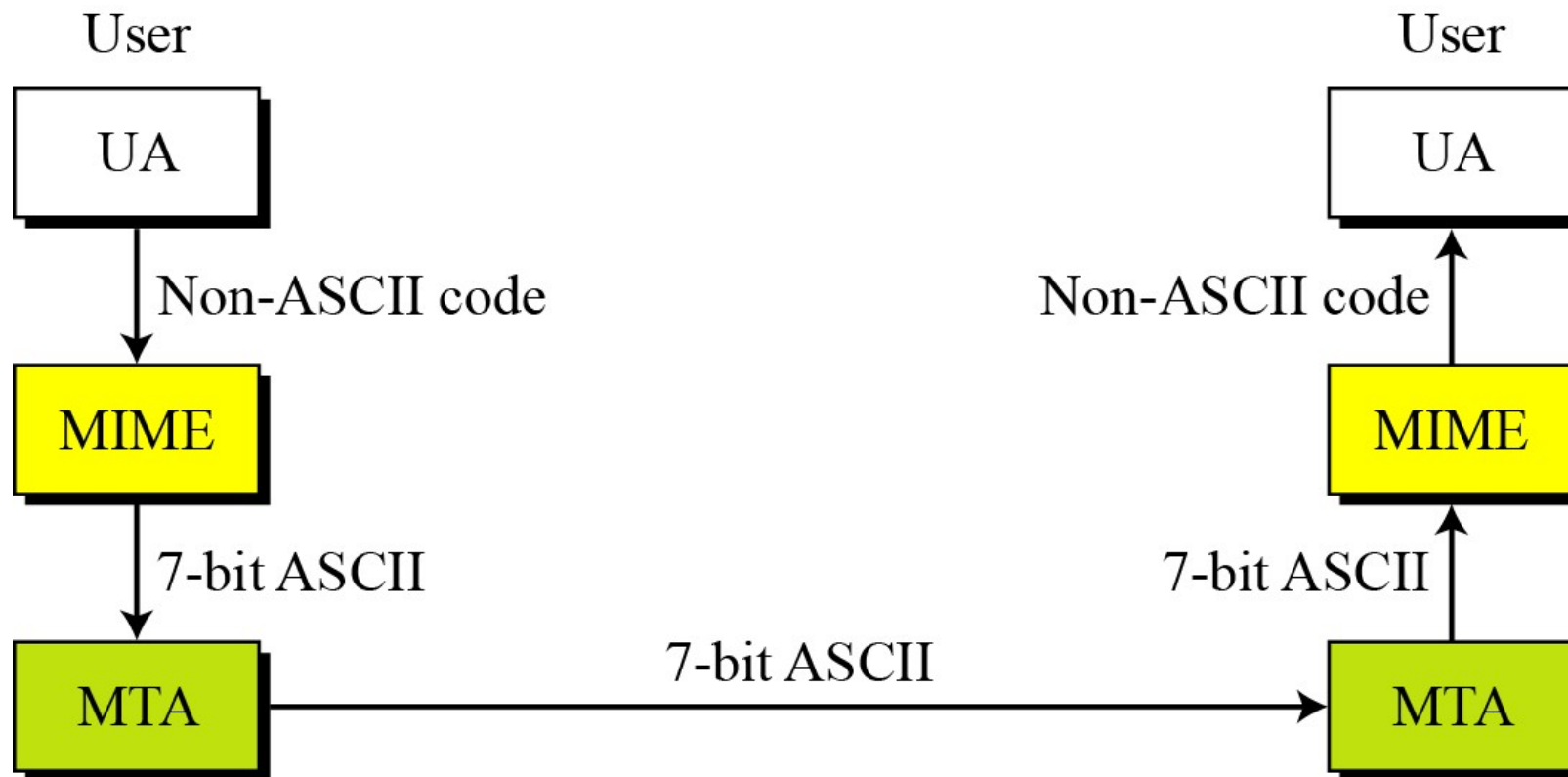


Figure *MIME header*

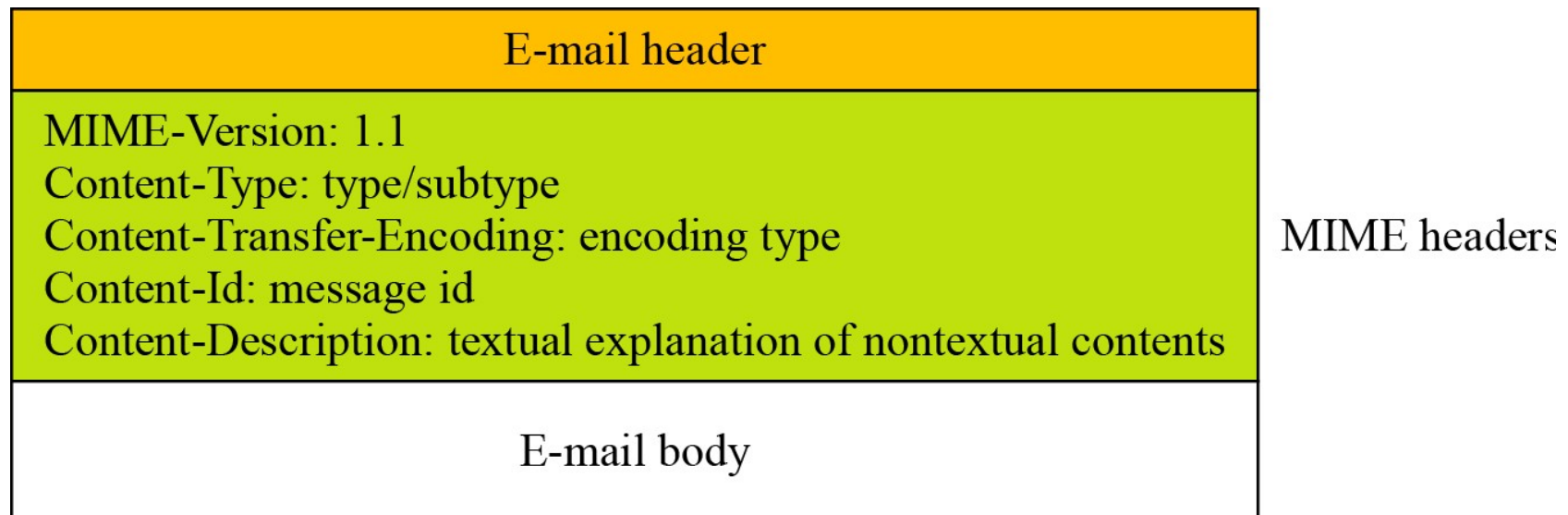
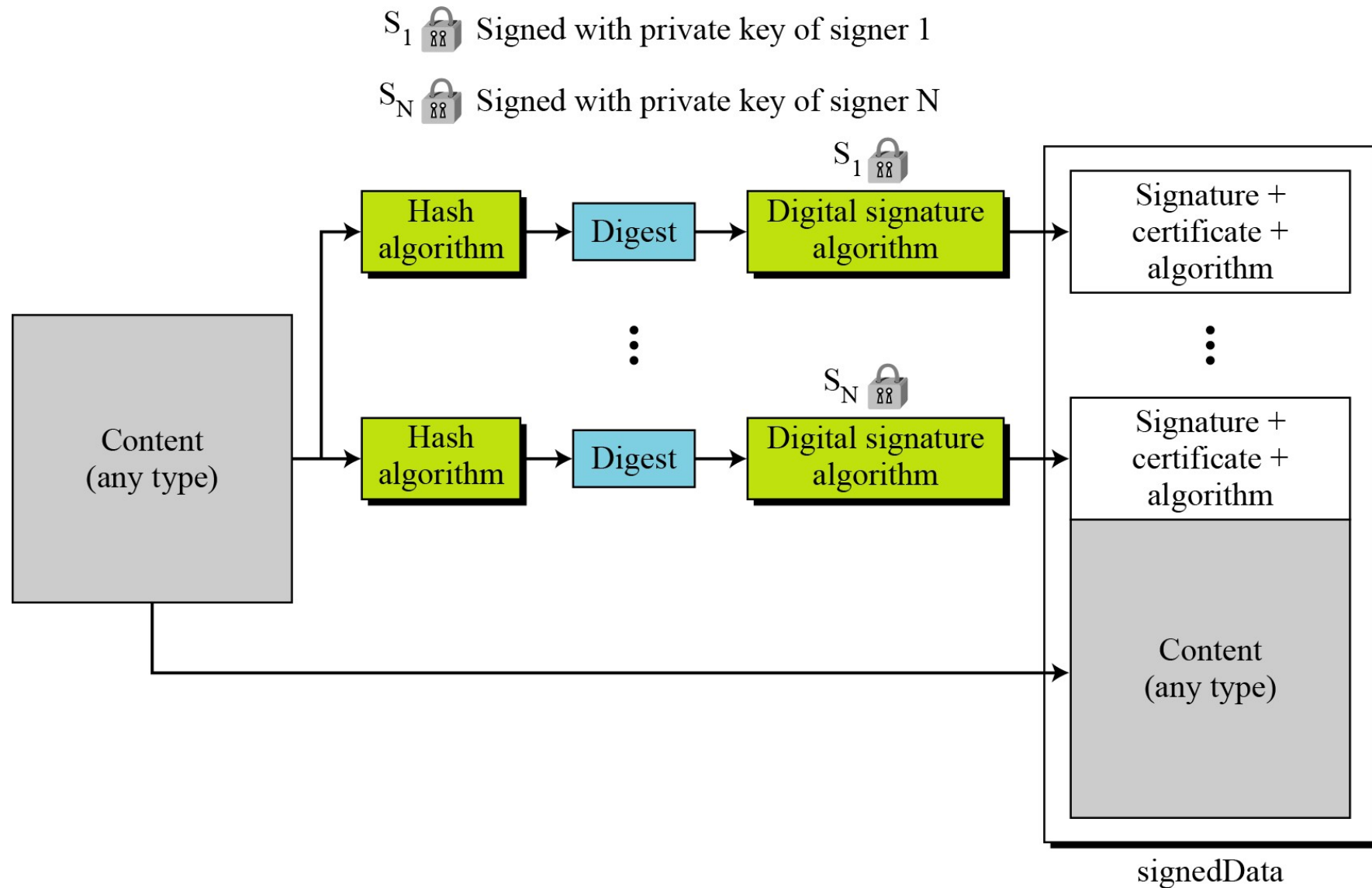


Table 16.14 *Data types and subtypes in MIME*

<i>Type</i>	<i>Subtype</i>	<i>Description</i>
Text	Plain	Unformatted.
	HTML	HTML format.
Multipart	Mixed	Body contains ordered parts of different data types.
	Parallel	Same as above, but no order.
	Digest	Similar to Mixed, but the default is message/RFC822.
	Alternative	Parts are different versions of the same message.
Message	RFC822	Body is an encapsulated message.
	Partial	Body is a fragment of a bigger message.
	External-Body	Body is a reference to another message.
Image	JPEG	Image is in JPEG format.
	GIF	Image is in GIF format.
Video	MPEG	Video is in MPEG format.
Audio	Basic	Single channel encoding of voice at 8 KHz.
Application	PostScript	Adobe PostScript.
	Octet-stream	General binary data (eight-bit bytes).

S/MIME

Figure *Signed-data content type*



Enveloped-data content type

R_1  Encrypted with public key of recipient 1

R_N  Encrypted with public key of recipient N

 Encrypted with session key

Session key created by
pseudorandom
generator

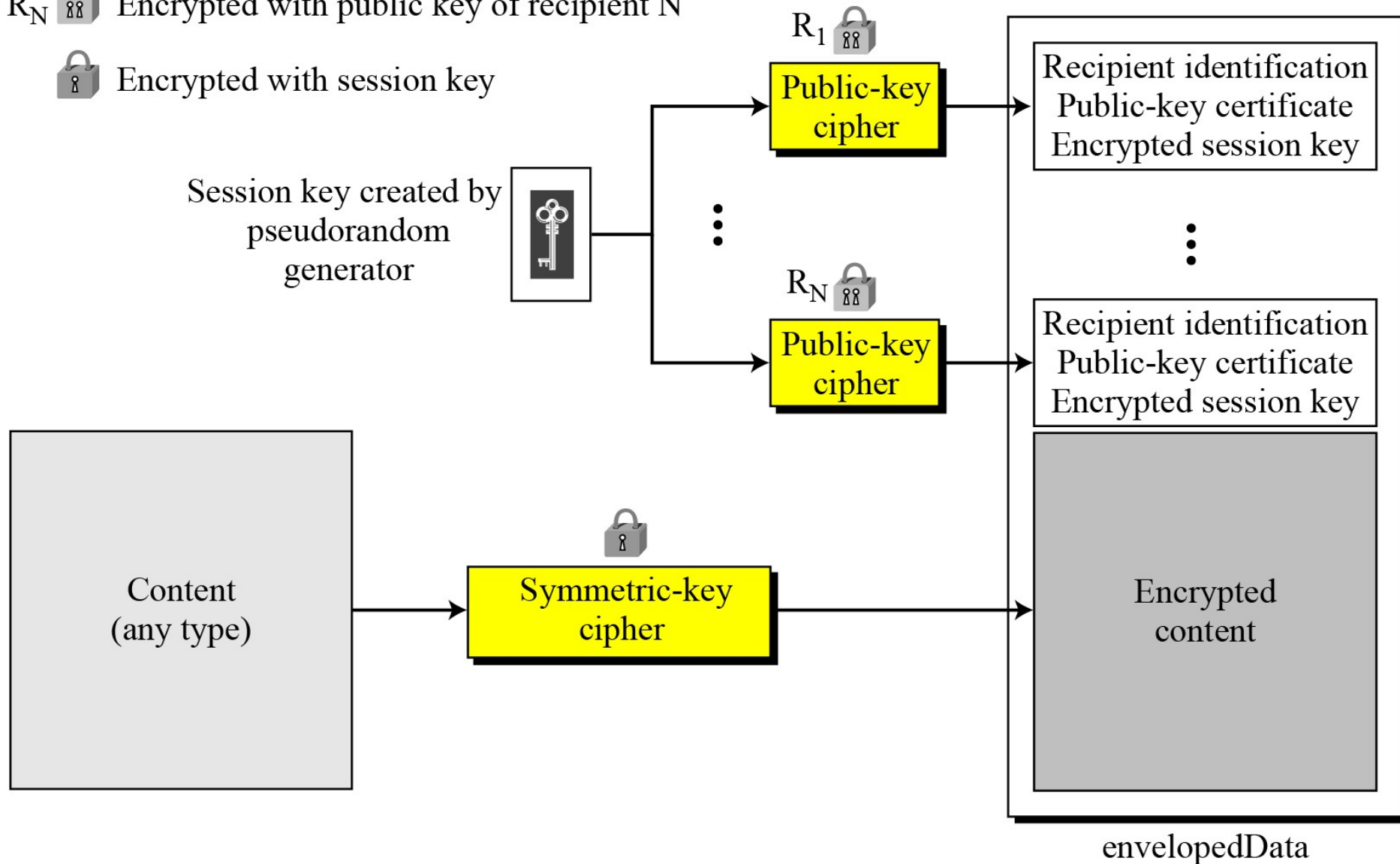


Figure *Digest-data content type*

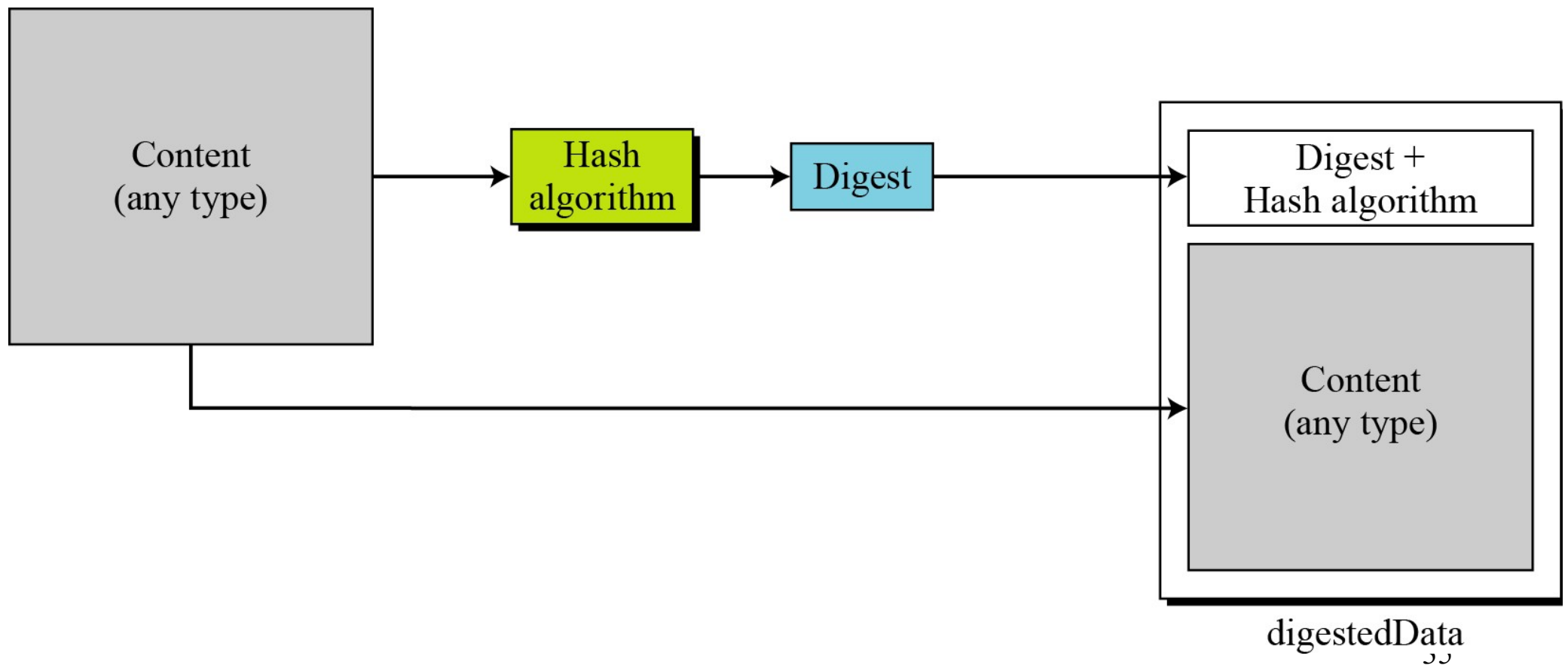


Figure *Authenticated-data content type*

R_1  Encrypted with public key of recipient 1

R_N  Encrypted with public key of recipient N

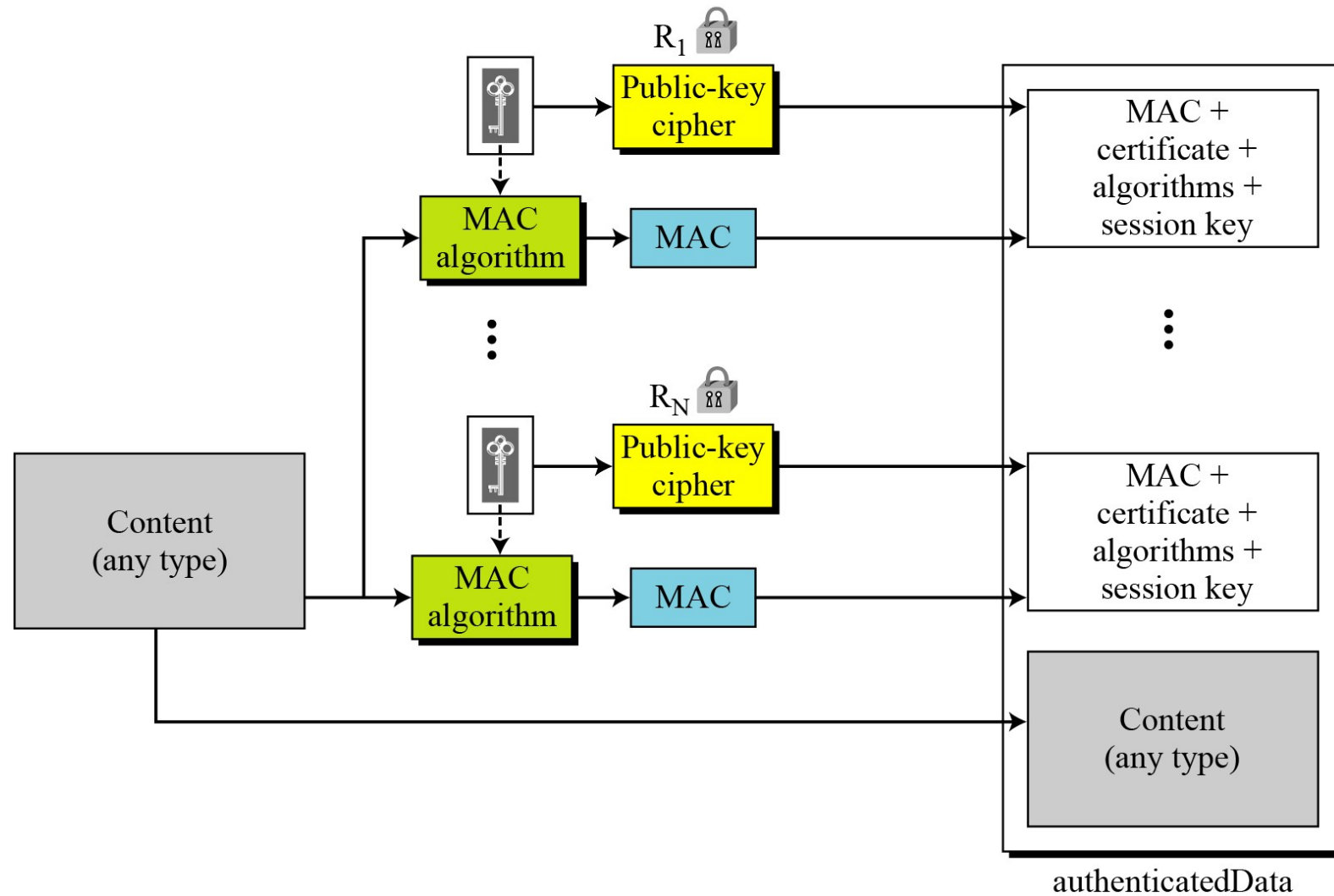


Table 16.17 *Cryptographic algorithm for S/MIME*

<i>Algorithm</i>	<i>Sender must support</i>	<i>Receiver must support</i>	<i>Sender should support</i>	<i>Receiver should support</i>
Content-encryption algorithm	Triple DES	Triple DES		1. AES 2. RC2/40
Session-key encryption algorithm	RSA	RSA	Diffie-Hellman	Diffie-Hellman
Hash algorithm	SHA-1	SHA-1		MD5
Digest-encryption algorithm	DSS	DSS	RSA	RSA
Message-authentication algorithm		HMAC with SHA-1		

S/MIME Functions

- **Enveloped Data:** Encrypted content and encrypted session keys for recipients.
- **Signed Data:** Message Digest encrypted with private key of "signer."
- **Clear-Signed Data:** Signed but not encrypted.
- **Signed and Enveloped Data:** Various orderings for encrypting and signing.

User Agent Role

- S/MIME uses Public-Key Certificates - X.509 version 3 signed by Certification Authority
- Functions:
 - **Key Generation** - Diffie-Hellman, DSS, and RSA key-pairs.
 - **Registration** - Public keys must be registered with X.509 CA.
 - **Certificate Storage** - Local (as in browser application) for different services.